

Locality-Aware Task Scheduling and Data Distribution for OpenMP

Muddukrishna, Jonsson & Brorsson · *Scientific Programming*, 2015 · NUMA systems and manycore processors

The hardware problem

Reading another node's memory costs 5–7× more than your own. On manycore chips the same gap now exists **inside one chip**, between cache banks.

The software problem

OpenMP's work-stealing scheduler balances load perfectly but is **blind to where data lives**. It will run a task on a core far from its memory.

Their fix

Let the programmer say how data is spread, then let the runtime place each task near its own data. Up to **+50% NUMA, +88% manycore**.

STEP 1 – SEQUENTIAL IMPLEMENTATION & PROFILING

100% of the time is in two kernels

Dense matrix multiply, **naive** vs **blocked**, both doing an identical $2n^3$ operations and verified to zero difference. Profiled with gprof at -O0, -O2, -O3.

gprof flat, -O0	% time	calls
matmul_naive	75.02	1
multiply_block	24.98	4096
everything else	0.00	—

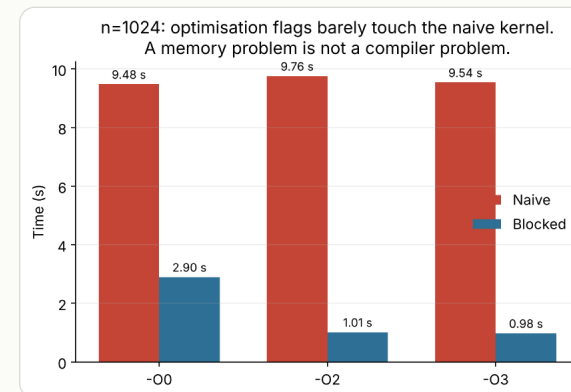
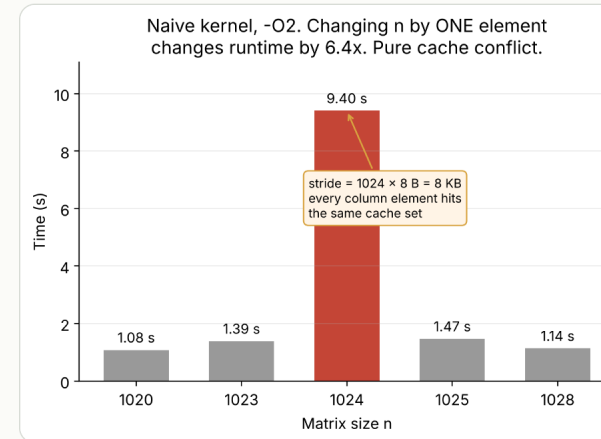
Sanity check: 4096 calls = $(1024/64)^3$. The counts are internally consistent, so the profile is trustworthy.

Blocking speedup at -O3

9.75x

Cache conflict cliff

6.4x



matmul.c · gcc 13.3 · n = 1024, block = 64

STEP 2 – IS THE HOTSPOT PARALLELIZABLE?

Yes for the outer blocks, no for the accumulation

Loop	Verdict	Reason
ii	Yes	Each value writes a disjoint set of rows of C. No two iterations touch the same output element, so there is no race.
jj	Yes	Disjoint columns of C. Together the (ii, jj) pairs give 256 independent tasks at n=1024, bs=64.
kk	No	Every iteration accumulates into the same block of C. Parallelising it directly is a write-write race; it would need private partial blocks or atomics, both costing more than they gain.
i, j, k	Not useful	Trip count is only the block size (64). Thread overhead would dominate. Left serial so the compiler can vectorise them.
fill_random	No	rand() carries internal state, so calls are order dependent. It is also 0% of runtime.

What we parallelize

```
#pragma omp task
  depend(in: A[ii], B[jj])
  depend(inout: C[ii+jj])
```

One task per block pair; the kk loop stays sequential inside it.

Why this maps onto the paper

Those **depend** clauses name exactly the data footprint the paper's Algorithm 1 needs to build its histogram *D* and place a task near its memory. Standard OpenMP uses them only for ordering and **throws the locality information away**. That gap is the paper's whole point.

256 independent tasks · verified against the sequential reference

OpenMP, and the alternatives fail for concrete reasons

CHOSEN

OpenMP

- The paper **is about** OpenMP's task scheduler. Rebuilding it elsewhere would not be reproducing it.
- Shared memory with a locality penalty is precisely the case it addresses.
- It already exposes the knobs: `depend`, `OMP_PROC_BIND`, `OMP_PLACES`, `first-touch`.
- Incremental: the sequential code stays valid, matching the paper's goal of an architecture-oblivious approach.

REJECTED

MPI

Giving every rank a private copy **destroys the effect we are studying**. Once remote data is an explicit message rather than a transparent load, we are measuring network bandwidth, not memory placement. MPI also has no runtime scheduler deciding where a task runs.

Right choice only if the matrices exceeded one node's RAM. They do not.

REJECTED

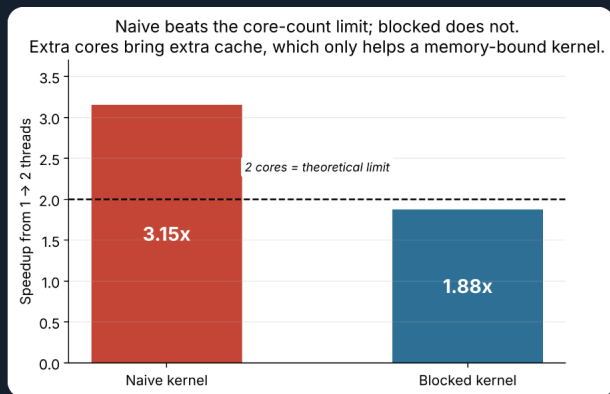
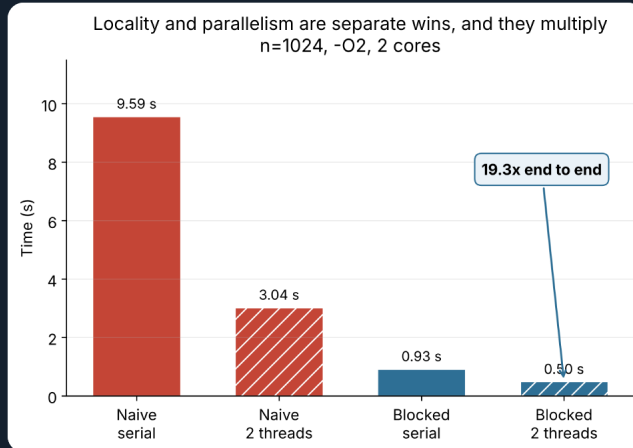
CUDA

A GPU has its own hierarchy, but it is **not NUMA and not a cache-bank topology**, so the paper's model of nodes, distances and home caches has no counterpart. Warp scheduling is fixed in hardware, so there is nothing for a locality-aware scheduler to influence.

The comparison would collapse into “GPU beats CPU”, which says nothing about the paper.

STEP 4 – COMPARATIVE RESULTS

Locality and parallelism multiply: 19.3× end to end



n = 1024	serial	2 thr	gain
naive	9.59 s	3.04 s	3.15×
blocked	0.93 s	0.50 s	1.88×

Superlinear, and it is evidence

The naive kernel gets **3.15× from 2 cores**, beating the core count. A second core brings a second cache, which only helps if you were memory bound. Blocked gets 1.88× because it had already fixed that.

Honest negative result

Work stealing, binding and first-touch land within **2.5% of each other**. Our machine has **one NUMA node and two cores**: there is no topology to exploit, so there is nothing for the paper's mechanism to win back. Its gains need a multi-socket machine.

Per-thread overhead is real: at one thread the task version runs at **0.97×** of sequential, pure task-creation cost with nothing to gain from it.